

Software Testing and Quality Assurance

Lecture 5

Fabian Fagerholm

fabian.fagerholm@aalto.fi



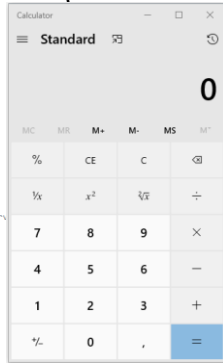
Aalto University
School of Science



Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Concepts

System Under Test (SUT): The software system being tested



```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Techniques for designing tests

Techniques for designing black-box tests

- Equivalence partitioning
- Boundary value analysis
- Robustness testing
- Decision table testing
- State transition testing
- Use case testing
- Scenario-based testing

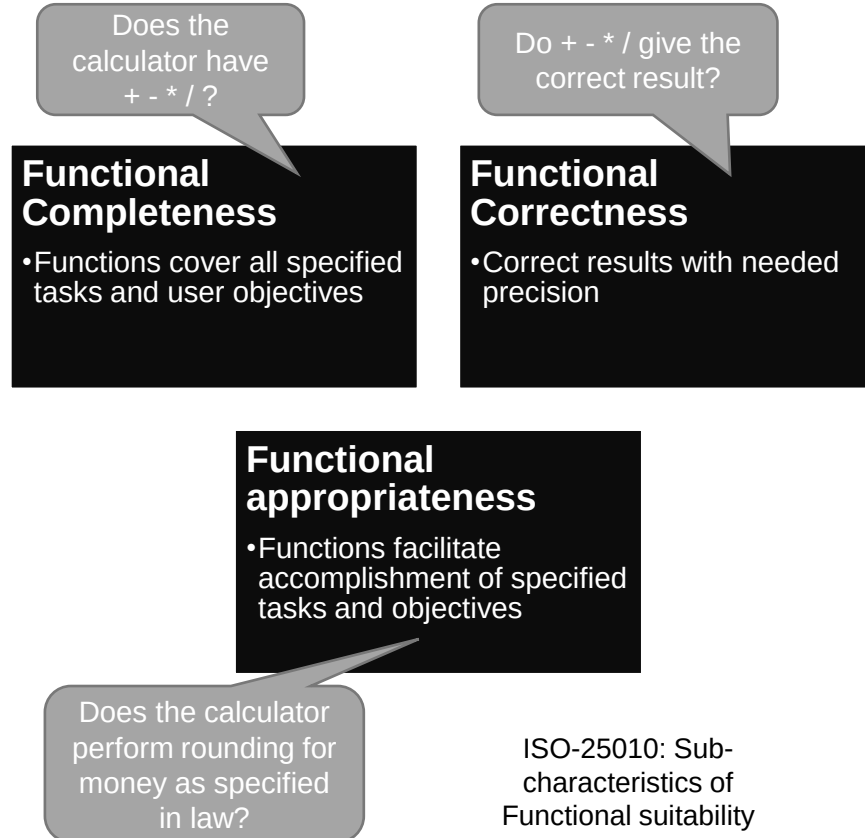
Techniques for designing white-box tests

- Control flow testing
- Statement coverage
- Path coverage
- Decision coverage
- Condition coverage
- Condition/decision coverage
- Modified condition/decision coverage

Functional suitability

- The essential purpose of the software system
- “Degree to which [system] provides functions that meet stated and implied needs when used under specified conditions” (ISO 25010)
 - Each individual function can be verified as existing or not existing
 - Specific properties can be specified for a function

Functionality is “just the beginning” of quality



Manual testing

Manual testing: motivation

- Preventing all faults is practically impossible
- Could all tests be automated?
 - Creating an oracle suitable for automation is often not possible
 - Automation can generally find only the most extreme and most visible faults
 - Automated test code can have faults; then manual intervention is needed
 - Automated tests need maintenance; they too incur costs

→ **Manual testing is needed**

Test automation and manual testing can complement each other.

A manual test may be a good starting point for designing an automated test.



Discussion

Manual testing: a human performs the test (possibly using some tools)

What benefits and challenges
do you identify with manual testing?

How could the challenges be reduced?

Manual testing

- “Human-present testing”
 - A human steers the testing and performs some essential testing actions
 - A human makes the judgement of whether the test passes or not
 - Some tools could be used to help
- Problems can arise with unsystematic manual testing:
 - Ad-hoc testing
 - Manual testing can be too slow
 - Not repeatable
 - Not reproducible
 - Not transferable



Photo: CC BY-SA

Systematic manual
testing

Scripted
testing

Exploratory
testing

Scripted testing

- The tester follows a script to perform tests
- **Rigid scripts**
- Flexible scripts
- Scripted testing can be as rigid or as flexible as necessary

BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

Functional area	Test Name	Test Steps	Test Data	Expected Results
Sign-up	Sign-in success	1. Enter a valid username and password 2. Click Login	Username: mary Password: Secret1\$	User is logged in successfully
Sign-up	Sign-in with incorrect password	1. Enter valid username 2. Enter invalid password 3. Click Login	Username: mary Password: wrong0	User is returned to login page and wrong username or password message is shown

Scripted testing

- The tester follows a script to perform tests
- Rigid scripts
- **Flexible scripts**
- Scripted testing can be as rigid or as flexible as necessary

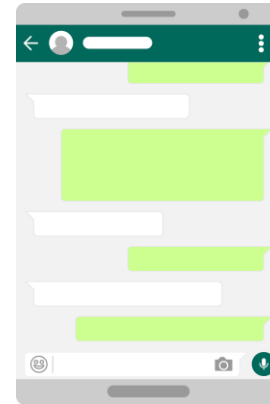
BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

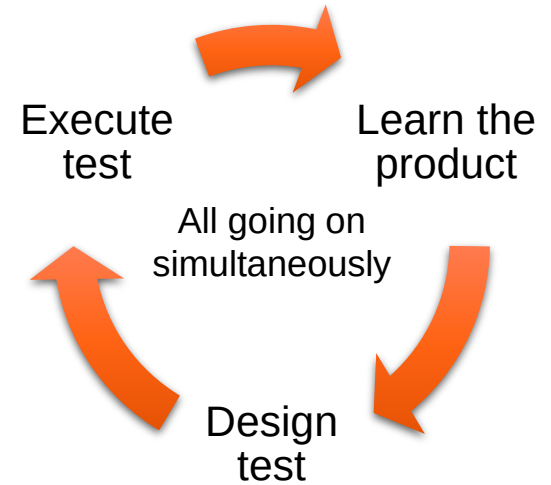
Chat bot test

1. Open the chat bot
2. Interact with the chat bot interface
3. Report findings



Exploratory testing

- No test scripts – but can use guiding plan
- Test documentation (cases, results) recorded as the test proceeds
- Tools can be used (debugger, screen recorder, etc.)
- Focus on *varying things* to explore the SUT
- Systematic guidance increases the chances of finding faults
- Requires skill and experience
- Management challenges
 - Keeping track of testing progress (especially with multiple testers)
 - Summarising the state and results of testing (e.g., for internal and external stakeholders)



Approaches to exploratory testing

- Exploratory testing in the small and in the large
 - Small decisions: the detailed choices the tester makes when exploring
 - Large decisions: choices about managing the overall exploration

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of "small" choices

User input

State

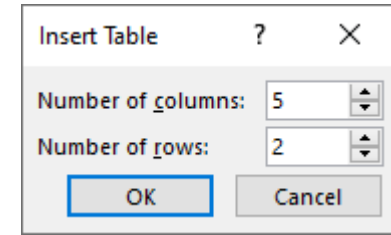
Code paths

User data

Environment

Choices: Input

- Input originates from outside the SUT and causes it to respond
- Legal (permitted) vs. illegal (not permitted) inputs
- Try values from different *equivalence classes*
 - Small positive integers (e.g. 1, 5)
 - Large positive integers (e.g. 40004, 65535)
 - Zero
 - Negative integers
 - Strings
- Filters: dropdowns, integer entry boxes
 - Can they be bypassed?

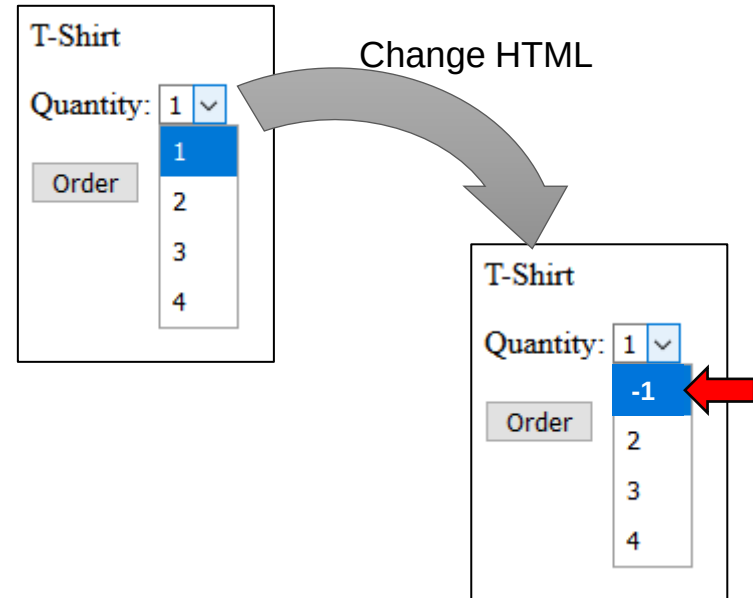


Insert Table ? X

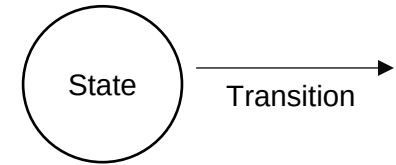
Number of columns: 5

Number of rows: 2

OK Cancel



Choices: State



- The state of the SUT is a coordinate in a state space that contains exactly one value for each internal data structure
- The state space can be extremely large and can only be partially tested
- When possible, start tests from *known states*

Keep track of inputs as they may reappear later, giving indications of internal state

Follow paths through the state space and explore both expected and unexpected paths

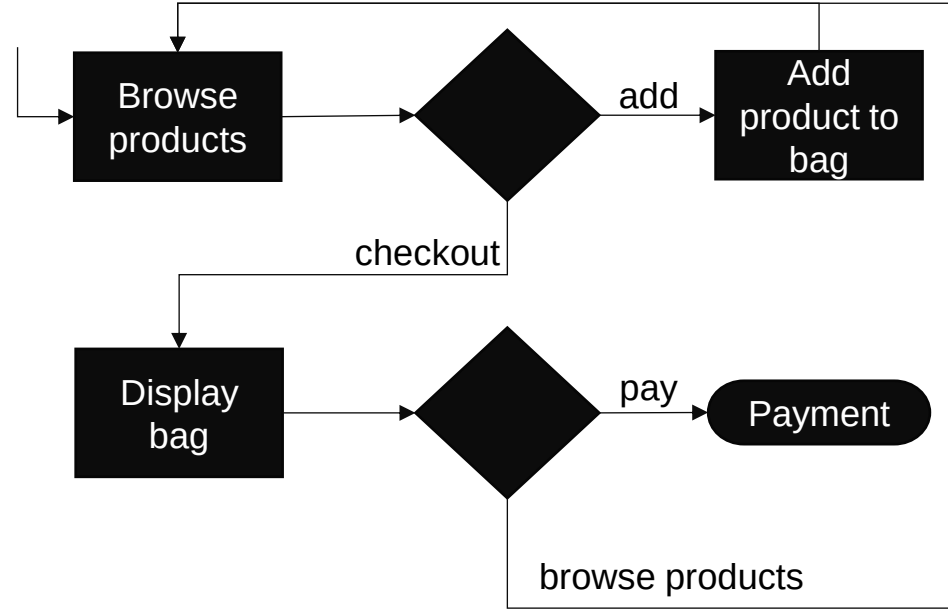
Check for correct data scope: is data *temporary* (persists over one session) or *persistent* (persists between sessions)

Test related inputs together (e.g. if testing a shopping voucher code, also have items in the cart)

Repeat state-changing actions to see if state is accumulative (e.g. a growing data structure)

Choices: Code paths

- Branching structures in the code create different paths that can be explored
- Code paths and state interact
- Without access to source code (or time to analyse it), it is difficult to know which paths exist and have been covered
- To some extent, tools can assist (e.g. web browser inspector)
- The tester must make a judgement based on available information



Loops together with branching structures can lead to very complicated code paths

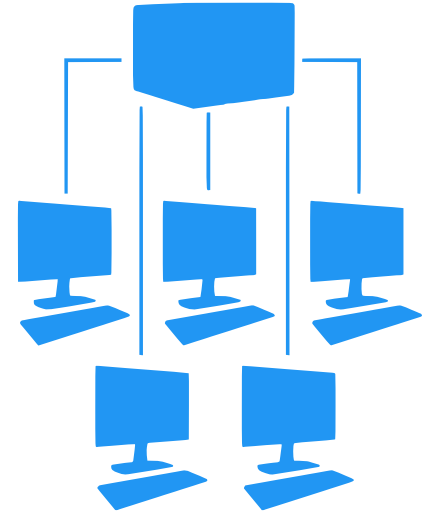
Choices: User data

- Often test data must be obtained or created to make certain tests possible
- Test data should be as similar as possible to data that users have, but:
- Real user data evolves over time
- Real user data often has unexpected relationships and structure
- Real user data is often large while it would be preferable to test with small, precise data
- Real user data is often confidential or has other restrictions that prevent its use in tests

→ Exploratory testing involves reasoned compromises regarding user data

Choices: Environment

- Differences in operating systems and OS configurations
- Other applications that might interact with the SUT
- Drivers, programs, files, and settings that might interact directly or indirectly with the SUT
- The network connection: its presence or absence, speed, and other properties
- The number of variations is extremely large and can often not be recreated



"Large" choices in exploratory testing

- Large choices concern the management of exploratory testing
- With many tests
- With many testers
- Considering the resources available (e.g. deadlines)
- Considering the quality requirements
- Considering efficiency and effectiveness of the testing process
- Considering needs to monitor and improve the testing process
- Considering the need to report on test results



Discussion

Manual testing: a human performs the test (possibly using some tools)

Analyse mycourses.aalto.fi and report on correctness and consistency issues as well as other observations

Perspective: a person who would like to apply to study at Aalto and is interested in software testing

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Sessions are
 - Chartered – they are associated with a mission regarding what to test or what problems are in focus
 - Uninterrupted – no significant interruptions
 - Typically between 45 minutes and 2 hours (this may vary and there is no strict rule)
- What happens during a session depends on the tester and the session charter

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Each session is debriefed
 - The session report is walked through
 - There can be a formal acceptance of the session report
 - Provide feedback and coaching to the tester
 - A debriefing can cover more than one session – they can be short
- The debriefing allows the test lead to
 - Understand how much can be done in one session
 - Understand how many sessions can be done per time
 - Provide a basis for effort predictions even though the testing is not planned in detail

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Session sheets
 - Function both as direction and reports
 - Reports are written in a standard format that is machine readable (as well as human-readable)
 - All reports are stored in a repository
 - The reports can be checked
 - E.g. if the report lists a data file, has the file been stored in the repository?
 - Metrics can be extracted from the reports
 - Are there bottlenecks preventing testing?
 - How many tests have we performed over the last 4 weeks?
 - How many tests can we expect to do next week?
 - (There is no one “standard” format)

Example session sheet

CHARTER

Analyze MapMaker's View menu functionality and report on areas of potential risk.

#AREAS

OS | Windows 2000

Menu | View

Strategy | Function Testing

Strategy | Functional Analysis

START

5/30/00 03:20 pm

TESTER

Jonathan Bach

Example session sheet

TASK BREAKDOWN

Book-keeping to this level is not always necessary – but can be done if needed.

#DURATION

short

#TEST DESIGN AND EXECUTION

65

#BUG INVESTIGATION AND REPORTING

25

#SESSION SETUP

20

#CHARTER VS. OPPORTUNITY

100/0

DATA FILES

#N/A

Keeping track of how much of the session followed the charter and how much was opportunistic.

Example session sheet

TEST NOTES

I touched each of the menu items, below, but focused mostly on zooming behavior with various combinations of map elements displayed.

View: Welcome Screen

Navigator

Locator Map

Legend

Map Elements

Highway Levels

Street Levels

Airport Diagrams

Zoom In

Zoom Out

Zoom Level

(Levels 1-14)

Previous View

Risks:

- Incorrect display of a map element.
- Incorrect display due to interrupted repaint.
- CD may be unreadable.
- Old version of CD may used.
- Some function of the product may not work at a certain zoom level.

Example session sheet

BUGS

#BUG 1321

Zooming in makes you put in the CD 2 when you get to a certain level of granularity (the street names level) -- even if CD 2 is already in the drive.

#BUG 1331

Zooming in quickly results in street names not being rendered.

#BUG <not_entered>

instability with slow CD speed or low video RAM. Still investigating.

ISSUES

#ISSUE 1

How do I know what details should show up at what zoom levels?

#ISSUE 2

I'm not sure how the locator map is supposed to work. How is the user supposed to interact with it?

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

- “Tour” is a metaphor that helps form an overall strategy and set specific goals
 - Aimless exploration: you might stumble upon something interesting
 - Specific goals and purpose: you will visit certain important landmarks
 - How much of each depends on the “tourist” – the character or persona whose *intent* guides the exploration
- Decomposition according to intent rather than the structure of the software
- Software functionality and features are visited more flexibly and in orders that differ from strict functional testing

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

- Districts
 - Business: startup/shutdown code, main features
 - Historical: legacy code
 - Tourist: features and functions that attract novice users
 - Entertainment: supportive features; complements the tour
 - Hotel: what the software does “at rest”
 - Seedy: areas of the software that are “off limits” to users

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of business district tours

- The Guidebook Tour
 - Read the manual and follow it to the letter
 - Variation: the Blogger's Tour – follow third-party advice
 - Variation: the Pundit's Tour – create test cases based on negative reviews
- The Intellectual Tour
 - Make the software work as hard as possible – try to break it
 - Look for inputs that cause problems (e.g. too large inputs, inputs that fool error checking)
 - But don't go overboard

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of hotel district tours

- The Rained-Out Tour
 - Start operations, then stop them
 - E.g., start a search, then cancel or start a print job, then cancel
 - Try different ways of cancelling – if there is no cancel button, do something else
 - Start, then restart without waiting for the operation to complete
- The Couch Potato Tour
 - Do the least work possible
 - Accept all defaults, leave fields blank, proceed to the next step without providing input, etc.
 - A strategy to exercise the default code paths

What does exploratory testing test?

- System-level and (in an exploratory fashion) acceptance test -level tests
- Can be inspired by (failed) automated tests – or can discover areas for further automated testing
- Exploratory testing is usually applied through the user interface of an application
 - Exploratory testing can find usability problems, but it is not the same as usability testing
 - Can focus on different quality characteristics and different stakeholders' perspectives
 - It can find important bugs quickly with little preparation
- There are also reports of exploratory testing used through APIs
 - Interactively for interpreted languages (e.g., Python)
 - Through exploratory scripts or small programs – works also for compiled languages
 - The purpose can still be to fulfil users' quality requirements – the “window” into the software is just different

Scientific evidence on exploratory testing

- Similar fault detection effectiveness as scripted testing (total number of defects found)
- Higher efficiency than scripted testing (defects found per time)
- Use of domain knowledge, the SUT, and customers, are important factors explaining the effectiveness
- Ability to provide rapid feedback reported as a benefit
- Limited possibilities to manage test coverage reported as a drawback

Itkonen, J.; Mäntylä, M. V. (2013). Are test cases needed? Replicated comparison between exploratory and test-case-based software testing. *Empirical Software Engineering*. 19 (2): 303–342.

Itkonen, J.; Mäntylä, M. V.; Lassenius, C. (2013). The Role of the Tester's Knowledge in Exploratory Software Testing. *IEEE Transactions on Software Engineering*. 39 (5): 707–724.

Itkonen, J.; Rautiainen, K. (2005). Exploratory testing: a multiple case study. 2005 International Symposium on Empirical Software Engineering, 2005.

Next steps

- Individual assignments in MyCourses
 - Quiz on path testing
 - Quiz on decision table testing
 - Unit testing 3
 - Related reading: Wang et al. (2024). Software Testing with Large Language Models: Survey, Landscape and Vision (see Materials)
 - Acceptance testing 2.1

Use the related reading
for more details.
Reserve enough time to
set up your environment.

Deadline: before next lecture (Tuesday by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺